1. Two Sum

Solved

Easy | Topics | Companies | Hint

Given an array of integers `nums` and an integer `target`, return *indices of the two numbers such that they add up to* `target`.

You may assume that each input would have ***exactly* one solution**, and you may not use the *same* element twice.

You can return the answer in any order.

Given:

nums = $[\underset{0}{a_0}, \underset{1}{a_1}, \underset{2}{a_2}, \ldots, \underset{n-1}{a_n}]$

Target (integer)

To find: ☆ Only on valid answers

Indices of 'nums' whose elements sum upto 'target'

Dry Run:

Q nums = $[\underset{0}{2}, \underset{1}{5}, \underset{2}{6}, \underset{3}{7}, \underset{4}{11}, \underset{5}{13}]$, target = 16

A $\{\cancel{0}, \{1, 4\}$

Approach 1 - Brute force

→ Traverse every element of 'nums' and for every element, traverse every element and check sum.

$[\overset{i}{2}, \overset{j}{5}, 6, 7, 11, 13]$ sum = 7

$[\overset{i}{2}, 5, \overset{j}{6}, 7, 11, 13]$ sum = 8

$[\overset{i}{2}, 5, 6, \overset{j}{7}, 11, 13]$ sum = 9

$[\overset{i}{2}, 5, 6, 7, \overset{j}{11}, 13]$ sum = 13

$[2, 5, 6, 7, 11, \overset{j}{13}]$ sum = 18

$[2, \overset{i}{5}, \overset{j}{6}, 7, 11, 13]$ sum = 11

$[2, \overset{i}{5}, 6, \overset{j}{7}, 11, 13]$ sum = 12

$[\underset{0}{2}, \overset{i}{\underset{1}{5}}, \underset{2}{6}, \underset{3}{7}, \overset{j}{\underset{4}{11}}, \underset{5}{13}]$ $\boxed{\text{sum} = 16 = \text{target}}$

ans = {1, 4}

~~Approa~~ Time complexity = $O(n^2)$

Space complexity = $O(1)$

<u>Approach 2</u> - <u>Hashmaps</u> (Optimal)

→ In the previous approach, we check all the elements, for every element in 'nums'

→ Now we use maps to store ~~whether~~ all the previously traversed elements with their indices.

→ When we traverse 'nums' we check whether the complement of the current element exist before it using map datastructure.

Complement = target − nums[i]

map = { } [2, 5, 6, 7, 11, 13] Complement = 14
0 1 2 3 4 5 (not in map)

map = {2 → 0} [2, 5, 6, 7, 11, 13] complement = 11
(not in map)

map = {2 → 0, 5 → 1} [2, 5, 6, 7, 11, 13] complement = 10
(not in map)

map = {2 → 0, 5 → 1, 6 → 2} [2, 5, 6, 7, 11, 13] Comp = 9
(not in map)

map = {2 → 0, 5 → 1, 6 → 2} [2, 5, 6, 7, 11, 13] comp = 5
0 1 2 3 4 5
(exists in map)

~~Final~~ ans = {1, 4}

Time complexity = O(n)

~~Sap~~ Space complexity = O(n)
(for map)

```cpp
class Solution {
public:
    vector<int> twoSum(vector<int>& nums, int target) {
        unordered_map<int, int> map;
        vector<int> ans;
        for(int i=0; i<nums.size(); i++){
            int complement=target-nums[i];
            if(map.find(complement)!=map.end() && map[complement]!=i){
                ans={map[complement], i};
                break;
            }
            map[nums[i]]=i;
        }
        return ans;
    }
};
```